

Requirements engineering

In the waterfall model,^[1] **requirements engineering** is presented as the first phase of the software development process. Later development methods, including the rational unified process (RUP) for software, assume that requirements engineering continues through a system's lifetime.

Requirements management, which is a sub-function of Systems Engineering practices, is also indexed in the International Council on Systems Engineering (INCOSE) manuals.

Activities

The activities involved in requirements engineering vary widely, depending on the type of system being developed and the organization's specific practice(s) involved.^[2] These may include:

1. Requirements inception or requirements elicitation – Developers and stakeholders meet; the latter are inquired concerning their needs and wants regarding the software product.
2. Requirements analysis and negotiation – Requirements are identified (including new ones if the development is iterative), and conflicts with stakeholders are solved. Both written and graphical tools (the latter commonly used in the design phase, but some find them helpful at this stage, too) are successfully used as aids. Examples of written analysis tools: use cases and user stories. Examples of graphical tools: Unified Modeling Language^[3] (UML) and Lifecycle Modeling Language (LML).
3. System modeling – Some engineering fields (or specific situations) require the product to be completely designed and modeled before its construction or fabrication starts. Therefore, the design phase must be performed in advance. For instance, blueprints for a building must be elaborated before any contract can be approved and signed. Many fields might derive models of the system with the LML, whereas others, might use UML. Note: In many fields, such as software engineering, most modeling activities are classified as design activities and not as requirement engineering activities.
4. Requirements specification – Requirements are documented in a formal artifact called a Requirements Specification (RS), which will become official only after validation. A RS can contain both written and graphical (models) information if necessary. Example: Software requirements specification (SRS).
5. Requirements validation – Checking that the documented requirements and models are consistent and meet the stakeholder's needs. Only if the final draft passes the validation process, the RS becomes official.
6. Requirements management – Managing all the activities related to the requirements since inception, supervising as the system is developed, and even until after it is put into use (e. g., changes, extensions, etc.)

These are sometimes presented as chronological stages although, in practice, there is considerable interleaving of these activities.

Requirements engineering has been shown to clearly contribute to software project successes.^[4]

Problems

One limited study in Germany presented possible problems in implementing requirements engineering and asked respondents whether they agreed that they were actual problems. The results were not presented as being generalizable but suggested that the principal perceived problems were incomplete requirements, moving targets, and time boxing, with lesser problems being communications flaws, lack of traceability, terminological problems, and unclear responsibilities.^[5]

Criticism

Problem structuring, a key aspect of requirements engineering, has been speculated to reduce design performance.^[6] Some research suggests that it is possible if there are deficiencies in the requirements engineering process resulting in a situation where requirements do not exist, software requirements may be created regardless as an illusion misrepresenting design decisions as requirements ^[7]

See also

- [List of requirements engineering tools](#)
- [Requirements analysis](#), requirements engineering focused in software engineering.
- [Requirements Engineering Specialist Group \(RESG\)](#)
- [International Requirements Engineering Board \(IREB\)](#)
- [International Council on Systems Engineering \(INCOSE\)](#)
- [IEEE 12207 "Systems and software engineering – Software life cycle processes"](#)
- [TOGAF \(Chapter 17\)](#)
- [Concept of operations \(ConOps\)](#)
- [Operations management](#)
- [Software requirements](#)
- [Software requirements specification](#)
- [Software Engineering Body of Knowledge \(SWEBOK\)](#)
- [Design specification](#)
- [Specification \(technical standard\)](#)
- [Formal specification](#)
- [Software Quality](#)
- [Quality Management](#)
- [Scope Management](#)

References

1. [Royce, W. W. \(1970\). *Managing the Development of Large Software Systems: Concepts and Techniques* \(http://www.serena.com/docs/agile/papers/Managing-The-Development-of-Large-Software-Systems.pdf\) \(PDF\). International Conference on Software Engineering \(ICSE\) '87. *Proceedings of the 9th international conference on Software Engineering*. pp. 1–9.](http://www.serena.com/docs/agile/papers/Managing-The-Development-of-Large-Software-Systems.pdf)

2. Sommerville, Ian (2009). *Software Engineering* (9th ed.). Addison-Wesley. ISBN 978-0-13-703515-1.
3. "Uncovering Requirements With UML Class Diagrams Part 1" (<http://tynerblain.com/blog/2008/03/06/requirements-class-diagrams-1/>). *tynerblain.com*. March 7, 2008. Retrieved March 14, 2018.
4. Hofmann, H.F.; Lehner, F. (2001). "Requirements engineering as a success factor in software projects". *IEEE Software*. **18** (4): 58–66. Bibcode:2001ISoft..18d..58H (<https://ui.adsabs.harvard.edu/abs/2001ISoft..18d..58H>). doi:10.1109/MS.2001.936219 (<https://doi.org/10.1109%2FMS.2001.936219>). ISSN 0740-7459 (<https://search.worldcat.org/issn/0740-7459>).
5. Méndez Fernández, Daniel; Wagner, Stefan (2015). "Naming the pain in requirements engineering: A design for a global family of surveys and first results from Germany". *Information and Software Technology*. **57**: 616–643. arXiv:1611.04976 (<https://arxiv.org/abs/1611.04976>). doi:10.1016/j.infsof.2014.05.008 (<https://doi.org/10.1016%2Fj.infsof.2014.05.008>). S2CID 1924926 (<https://api.semanticscholar.org/CorpusID:1924926>).
6. Ralph, Paul; Mohanani, Rahul (May 2015). "Is Requirements Engineering Inherently Counterproductive?" (<https://www.researchgate.net/publication/272793687>). IEEE. doi:10.13140/2.1.3831.6321 (<https://doi.org/10.13140%2F2.1.3831.6321>). {{cite journal}}: Cite journal requires |journal= (help)
7. Ralph, P. (September 2013). "The illusion of requirements in software development". *Requirements Engineering*. **18** (3): 293–296. arXiv:1304.0116 (<https://arxiv.org/abs/1304.0116>). Bibcode:2013AIPC.1516..293R (<https://ui.adsabs.harvard.edu/abs/2013AIPC.1516..293R>). doi:10.1007/s00766-012-0161-4 (<https://doi.org/10.1007%2Fs00766-012-0161-4>). S2CID 11499083 (<https://api.semanticscholar.org/CorpusID:11499083>).

External links

- *Systems and software engineering -- Life cycle processes --Requirements engineering*. 2011. pp. 1–94. doi:10.1109/IEEESTD.2011.6146379 (<https://doi.org/10.1109%2FIEEESTD.2011.6146379>). ISBN 978-0-7381-6591-2. ("This standard replaces IEEE 830–1998, IEEE 1233–1998, IEEE 1362-1998 - <https://standards.ieee.org/ieee/29148/5289/>")
 - [Systems Engineering Body of Knowledge](http://sebokwiki.org/) (<http://sebokwiki.org/>)
 - [Requirements Engineering Management Handbook](https://www.faa.gov/aircraft/air_cert/design_approvals/air_software/media/AR-08-32.pdf) (https://www.faa.gov/aircraft/air_cert/design_approvals/air_software/media/AR-08-32.pdf) by FAA
 - [International Requirements Engineering Board \(IREB\)](https://ireb.org) (<https://ireb.org>)
 - [IBM Rational Resource Library](https://web.archive.org/web/20140310072616/http://spectrum.m.ieee.org/static/ibm-rational-resource-library) (<https://web.archive.org/web/20140310072616/http://spectrum.m.ieee.org/static/ibm-rational-resource-library>) by IEEE Spectrum
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Requirements_engineering&oldid=1358652691"